ML-BASED TEST QUALITY PREDICTION & REQUIREMENTS TRACEABILITY MATRIX (RTM)

R26-SE-039
Rashani K.G.M – IT22182678

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

January 2026

INTRODUCTION

Software testing is a critical phase in the software development lifecycle, consuming approximately 30-40% of project resources. Despite this investment, organizations face persistent challenges in ensuring test quality and maintaining traceability between requirements and tests. Two fundamental problems plague modern Quality Assurance (QA) processes: the inability to predict test effectiveness before execution, and the lack of automated mechanisms to track which requirements are covered by which tests.

The Test Quality Prediction Problem: Currently, QA teams must execute tests to discover their effectiveness. Poorly written tests with weak assertions, inadequate coverage, or missing boundary cases waste execution time and create false confidence in software quality. There exists no reliable method to predict, before execution, whether a test will actually detect bugs. This leads to inefficient test suites where low-quality tests consume resources while providing minimal value.

The Traceability Problem: Requirements Traceability Matrices (RTMs) are essential documents that map requirements to test cases, ensuring complete test coverage. However, in practice, RTMs are created manually, requiring 2-3 hours per project, and become outdated the moment requirements or tests change. Manual maintenance is error-prone, leading to coverage gaps where requirements remain untested, or redundant tests where multiple tests cover the same requirement without adding value.

This research addresses both challenges through a unified, ML-powered approach. The proposed system will:

1. **Predict Test Quality Before Execution:** ML models assign a 0–100 quality score to test cases based on coverage, assertions, boundary testing, and error handling. Tests scoring below 60 are automatically rejected.
2. **Automatically Generate and Maintain RTMs:** The system extracts requirements, links acceptance criteria to test cases using semantic analysis, maps tests to code through coverage analysis, and automatically updates the RTM.
3. **Detect Coverage Gaps Intelligently:** Analyzes the RTM to identify untested or partially tested requirements, detect missing error paths, and recommend additional tests.
4. **Optimize Test Portfolios:** Identifies redundant and critical tests using RTM insights and recommends improvements to optimize execution time.

These capabilities transform QA from manual, reactive processes into automated, proactive workflows, providing full requirement coverage visibility and optimized test suites.

METHODOLOGY

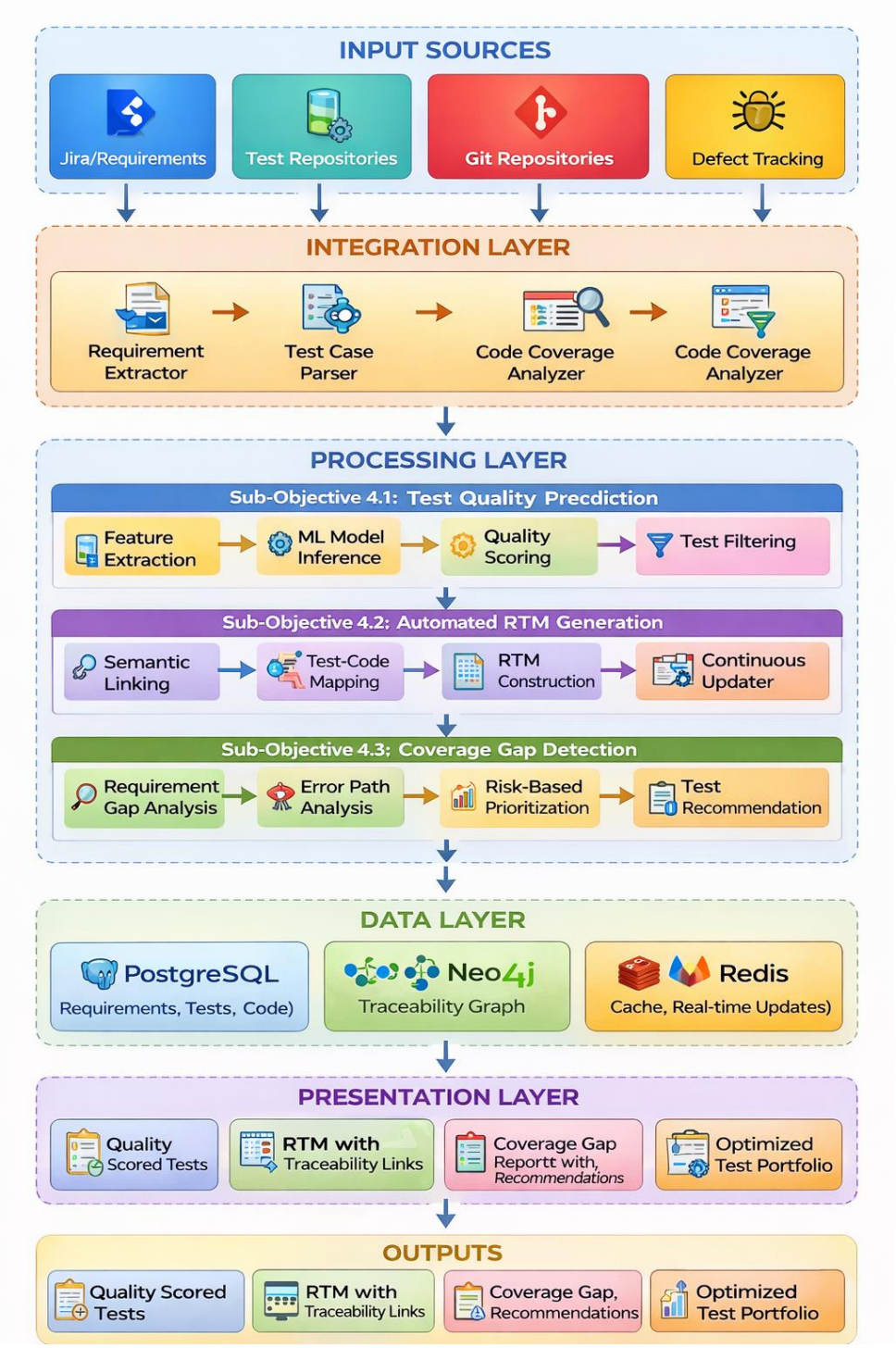
This research adopts a Design Science Research (DSR) approach to design and evaluate an intelligent framework for ML-based Test Quality Prediction and automated Requirements Traceability Matrix (RTM) generation. Design Science Research is appropriate because the objective of this study is to develop a technological solution that improves software testing quality, traceability, and test optimization in modern software development environments.

• The proposed solution addresses challenges in maintaining high-quality test suites, generating accurate traceability links, identifying testing gaps, and optimizing test portfolios within Agile and CI/CD environments. The system will analyze requirements, test cases, code artifacts, and historical defect data using machine learning and NLP techniques to predict test quality, automatically generate RTMs, detect coverage gaps, and recommend optimization strategies for improving software quality assurance.

• The research process consists of the following stages:

1. Collection of software development artifacts (requirements, test cases, code, and defect data)
2. Data preprocessing and feature engineering for test quality analysis
3. Machine learning model development for test quality prediction
4. Automated RTM generation using NLP and semantic similarity analysis
5. Coverage gap detection and risk-based analysis
6. Test portfolio optimization and recommendation generation
7. System integration, visualization, and CI/CD support
8. Experimental evaluation and validation of system performance

SYSTEM ARCHITECTURE



EXPECTED OUTCOMES

• **Test Quality Prediction Accuracy:** Achieving high accuracy, precision, recall, and F1-scores in predicting the quality of automated test cases using supervised machine learning models.

• **Automated RTM Generation Efficiency:** Successfully generating accurate traceability links between requirements, test cases, and code modules using NLP and semantic similarity techniques.

• **Improved Requirement Coverage:** Identifying untested and partially tested requirements, leading to better test coverage and reduced risk of missing critical functionalities.

• **Coverage Gap Detection:** Proactive detection of missing test scenarios, error-handling paths, and uncovered acceptance criteria through automated gap analysis.

• **Test Portfolio Optimization:** Reducing redundant and low-value test cases while maintaining required coverage and improving regression testing efficiency.

• **Reduction in Manual QA Effort:** Minimizing the manual effort required for maintaining RTMs, analyzing test quality, and identifying testing gaps in software projects.

• **Enhanced Decision-Making for QA Teams:** Providing intelligent recommendations for test improvements, prioritization, and optimization based on ML-driven insights.

• **CI/CD Integration Support:** Delivering a modular and scalable solution that can be integrated with modern DevOps and CI/CD environments for continuous quality monitoring and automated traceability updates.

REPO LINKS (GITHUB)

https://github.com/Nextgen-QA-Sliit/Quality-Prediction-and-Requirements-Traceability-Matrix